

Marwadi University

BCA Semester – 3

05BC3301 – Database Management System -2 (DBMS-2)

Unit – 2: Managing Cursor



Unit – 2: Managing Cursor

Overview of Cursor, Cursor Types, Implicit Cursor, Cursor Attributes, Writing Explicit Cursors (Cursor declaration, opening, fetching data from cursor, closing), Cursor FOR loop. Parameterized Cursor.

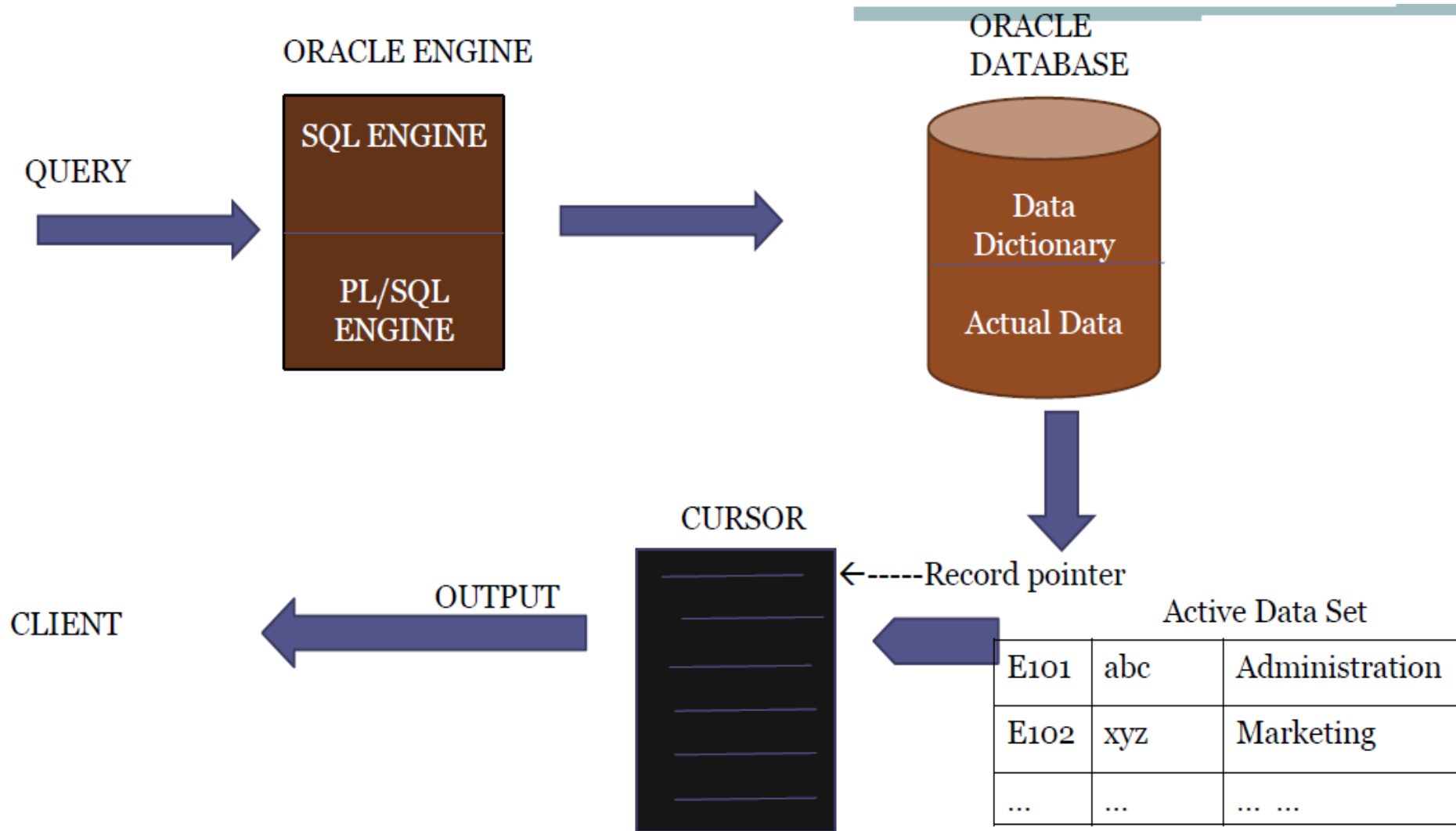
Topics

1. Overview of Cursor
2. How Cursor Works?
3. Introduction to Cursor
4. Need for Cursors?
5. Types of Cursors
 1. Implicit Cursors
 2. Explicit Cursors
6. Comparison between Implicit and Explicit Cursors OR
Difference between Implicit and Explicit Cursors
7. Cursor For Loop
8. Parameterized Cursor
9. Advantages and Disadvantages of Cursor

Overview of Cursor

- A cursor is a temporary work area created in the system memory when a SQL statement is executed.
- A cursor contains information on a select statement and the rows of data accessed by it.
- This temporary work area is used to store the data retrieved from the database, and perform intermediate operations before output is displayed to the client.
- A cursor can hold more than one row, but can process only one row at a time.

How Cursor Works?

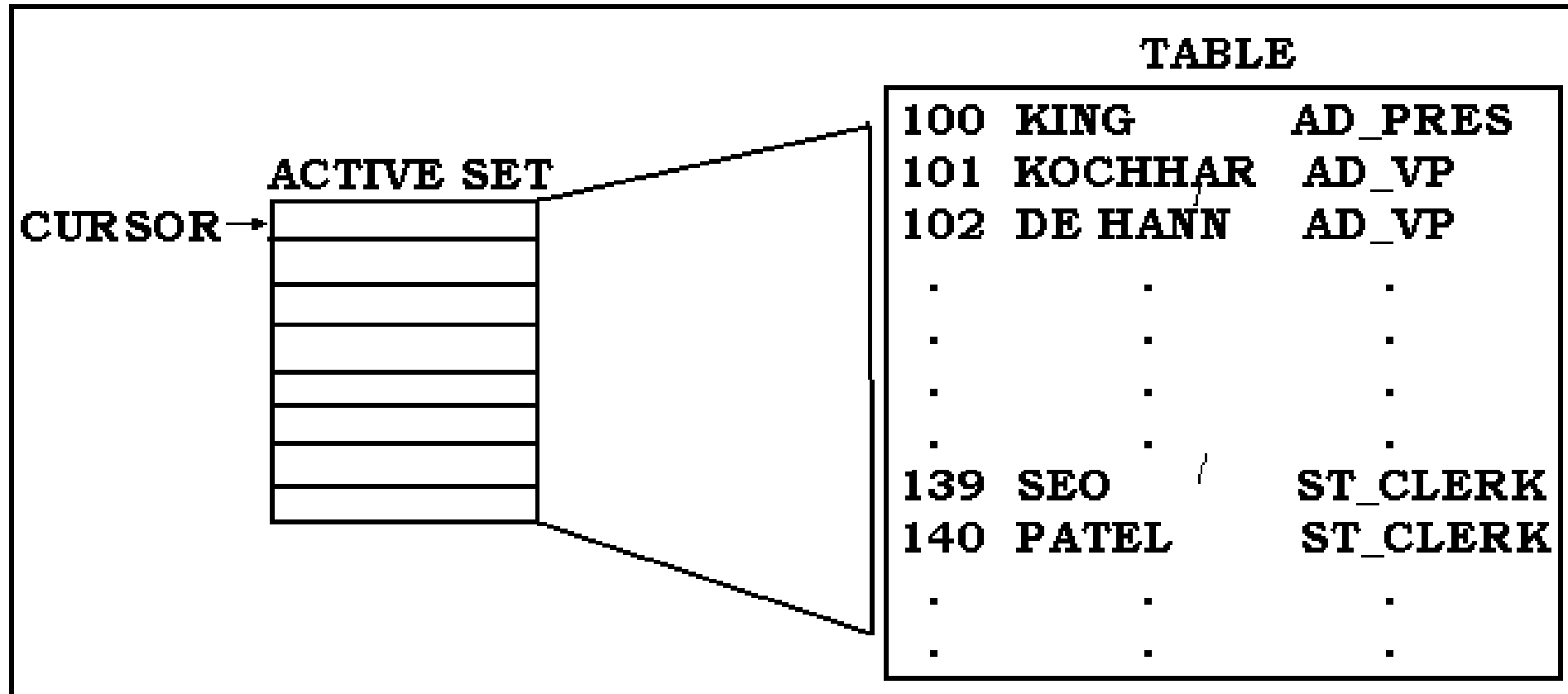


Introduction to Cursor

- Cursor is a memory area which store records written by SQL query. It is mainly used to perform operations on more then one row in PL block.
- The set of rows returned by a cursor is called an Active Data Set (Result Set). The row that is being processes is called the Current Row. The Current Row inside the Active Data Set (Result Set) is identified by the cursor, which allows the individual processing of each of them.
- The data that is stored in the cursor is called the **Active Data Set**.

Introduction to Cursor

- Example of Active Data Set



Need for Cursors?

- We cannot use sub-programs of PL/SQL with a simple select statement to retrieve more than one row.

SELECT statement in procedure returning 1 row

row

ORACLE processes procedure without error

SELECT statement in procedure returning more than 1 row

row 1

row 2

row 3

ORACLE returns error

- So, PL/SQL requires a special compatibility to retrieve and process more than one row

Need for Cursors?

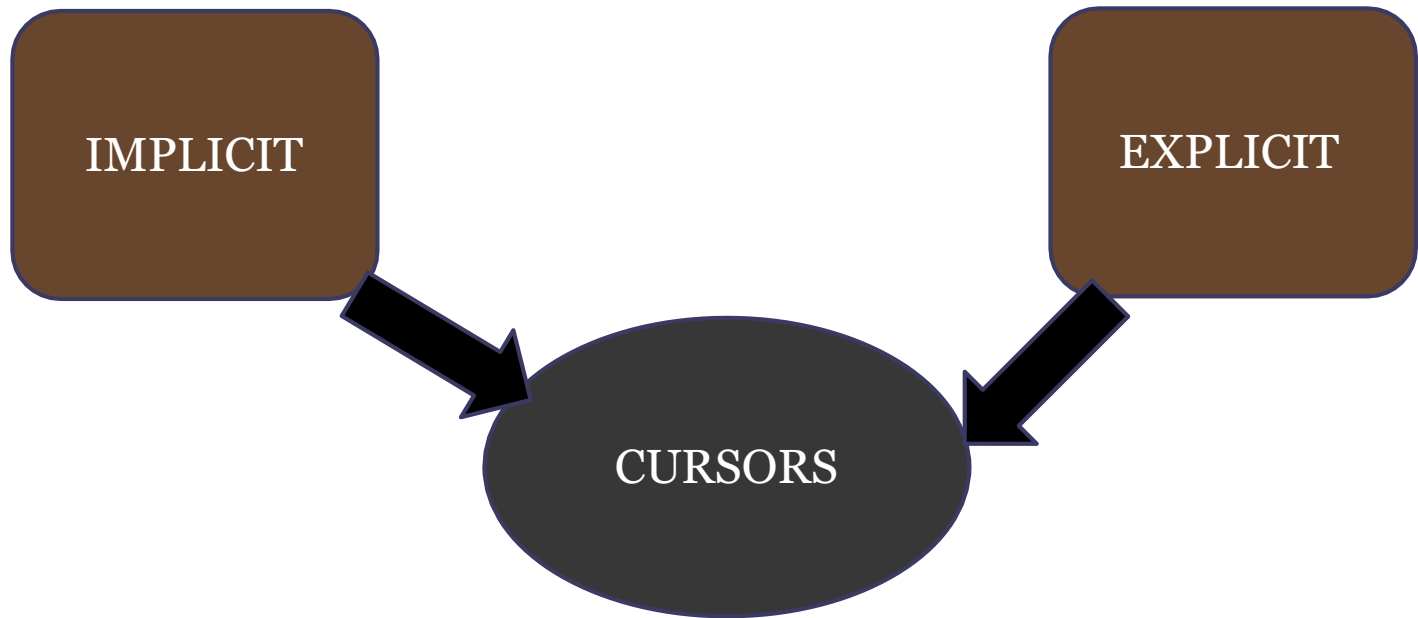
- Cursor is a mechanism that provides a way to select multiple rows of data from the database and then process each row individually inside a PL/SQL program.



- The cursor first points at row1 and once it is processed it then advances to row2 and so on.

Types of Cursors

- There are basically two types of cursors are there:
1. Implicit Cursor, and 2. Explicit Cursor.



Types of Cursors

- **Implicit Cursor:**
- These are created by default when DML statements like, INSERT, UPDATE, and DELETE statements are executed.
- The user is not aware of this happening & will not be able to control or process the information.
- When an implicit cursor is working, DBMS performs the open, fetches and close operations automatically.

Implicit Cursors

▪ Implicit Cursor (Cursor Attributes)

Attribute Name	Description
%ISOPEN	Returns TRUE if cursor is open, FALSE if cursor is closed. SQL%ISOPEN always returns FALSE.
%FOUND	Returns TRUE if successful fetch has been executed, FALSE if no row was returned. SQL%FOUND is used to access it.
%NOTFOUND	Return TRUE if no row was returned, FALSE if successful fetch has been executed. SQL%NOTFOUND is used to access it.
%ROWCOUNT	Returns the number of rows affected by the query. SQL%ROWCOUNT is used to access it.

Implicit Cursors

- **Implicit Cursor (Example):**
- Write a pl/sql program to update the salary of customer by Rs 500.
- The data of customers table before update operation:

Select * from customers;

+----+-----+----+-----+-----+				
ID	NAME	AGE	ADDRESS	SALARY
+----+-----+----+-----+-----+				
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
+----+-----+----+-----+-----+				

Implicit Cursors

- **Implicit Cursor (Example):**

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    total_rows number(2):=0;
```

```
BEGIN
```

```
    UPDATE customers SET salary = salary + 500;
```

```
    IF sql%notfound THEN
```

```
        dbms_output.put_line('no customers selected');
```

```
    ELSIF sql%found THEN
```

```
        total_rows:=sql%rowcount;
```

```
        dbms_output.put_line(total_rows || 'customers salary updated');
```

```
    END IF;
```

```
END;
```

```
/
```

Implicit Cursors

- **Implicit Cursor (Example Output):**
- 6 customers salary updated.
- PL/SQL procedure successfully completed.
- Data of customer table after update operation:

Select * from customers;

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2500.00
2	Khilan	25	Delhi	2000.00
3	kaushik	23	Kota	2500.00
4	Chaitali	25	Mumbai	7000.00
5	Hardik	27	Bhopal	9000.00
6	Komal	22	MP	5000.00

Types of Cursors

- **Explicit Cursor:**

- Explicit cursors are programmer defined cursors for gaining more control over the context area.
- An explicit cursor should be defined in the declaration section of the PL/SQL Block.
- It is created on a SELECT Statement which returns more than one row.

Explicit Cursors

▪ Explicit Cursor (Cursor Attributes)

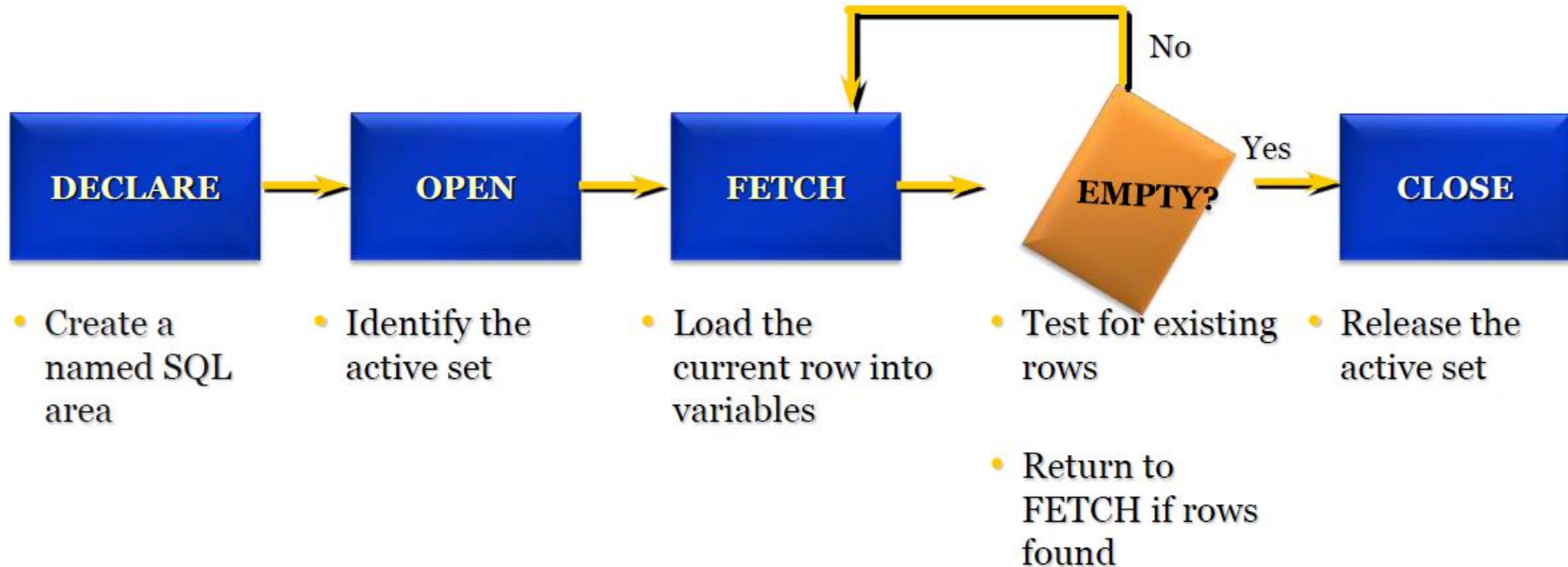
Attribute Name	Description
%ISOPEN	Returns TRUE if cursor is open, FALSE if cursor is closed. CursorName%ISOPEN is used to access it.
%FOUND	Returns TRUE if successful fetch has been executed, FALSE if no row was returned. CursorName%FOUND is used to access it.
%NOTFOUND	Return TRUE if no row was returned, FALSE if successful fetch has been executed. CursorName%NOTFOUND is used to access it.
%ROWCOUNT	Returns the number of rows affected by the query. CursorName%ROWCOUNT is used to access it.

Explicit Cursors

- **Explicit cursor involves four steps:**
 1. Declaring the cursor for initializing in the memory
 2. Opening the cursor for allocating memory
 3. Fetching the cursor for retrieving data
 4. Closing the cursor to release allocated memory

Explicit Cursors

- Explicit cursor involves four steps:



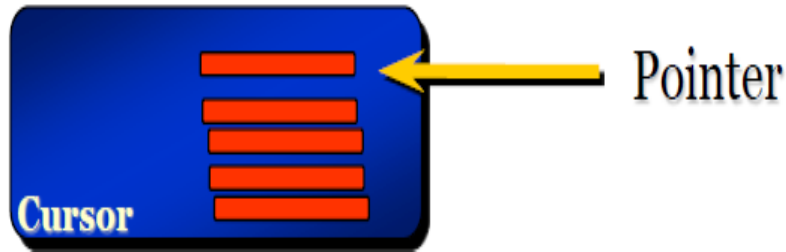
Explicit Cursors

- **Step 1: Cursor Declaration**
- Declared as a variable in the same way as standard variables.
- Identified as cursor type
- SQL included

E.g.

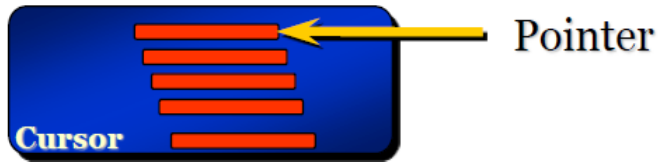
```
CURSOR cur_emp IS  
  
SELECT emp_id, name, grade  
FROM employee;
```

Explicit Cursors



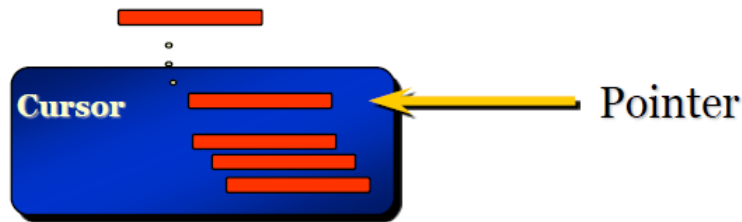
- **Step 2: Open the Cursor**
- Defines a private SQL area named after the cursor name.
- Executes the query associated with the cursor.
- Creates the Active Data Set.
- Sets the cursorrow pointer in the Active Data Set to the first record.

Explicit Cursors



Open the cursor.

Fetch a row from the cursor.



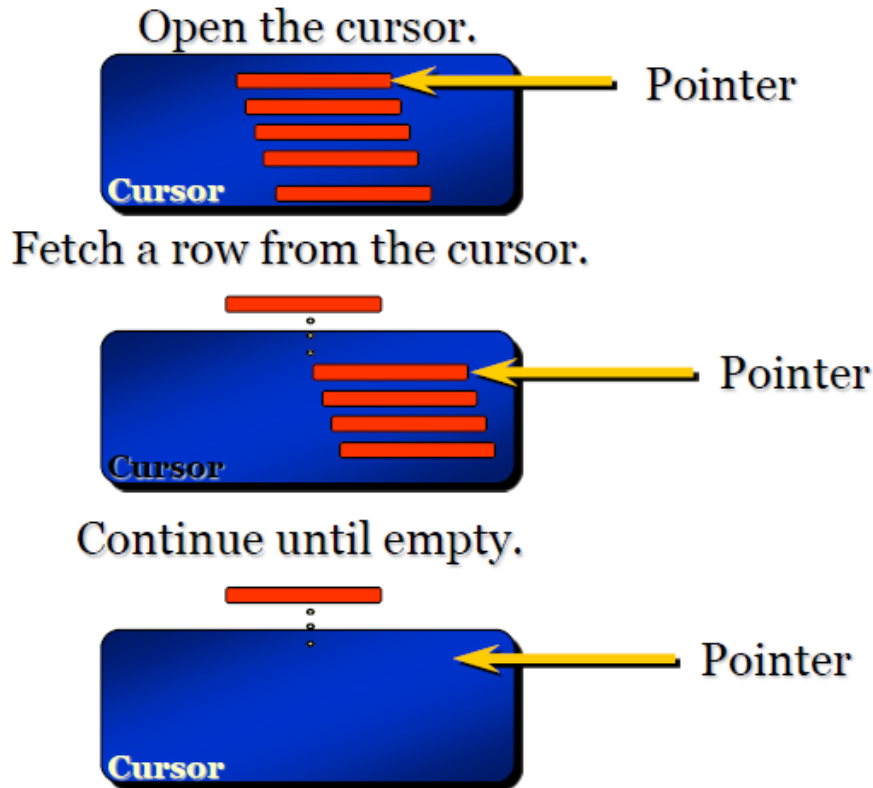
- **Step 3: Fetch the records**
- Fetch statement is placed inside a Loop ... End Loop construct.
- It takes the data into memory variables and process them one by one.

Explicit Cursors

- **Step 3: Fetch the records**

- Fetch command continue to fetch the records until the end of file encounters.

- Loop is continued until Active Data Set becomes empty

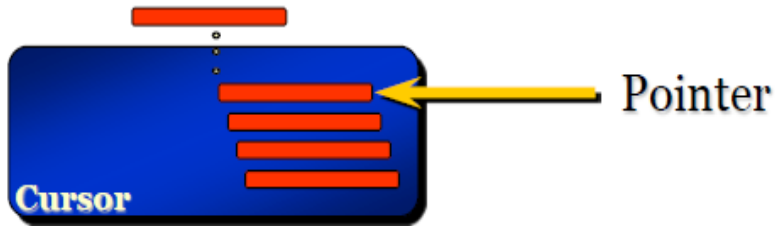


Explicit Cursors

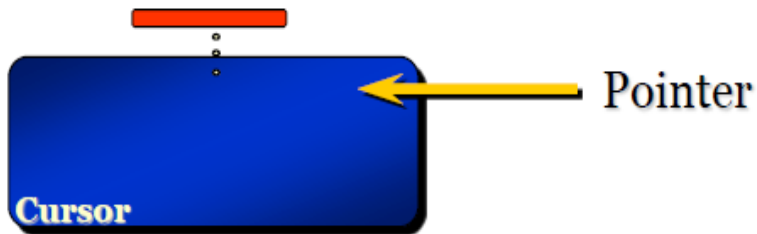
Open the cursor.



Fetch a row from the cursor.



Continue until empty.



Close the cursor.



- **Step 4: Close the Cursor**
- After the FETCH loop exits, the cursor must be closed by Close statement.
- At the end it releases the used memory

Explicit Cursors

- **Explicit Cursor (Syntax):**
- **Syntax for cursor declaration**

```
CURSOR cursor_name IS  
    select_statement;
```

CURSOR <CURSOR_NAME> IS <SELECT STATEMENT>;

For Example:

```
CURSOR CUR_SAL IS  
SELECT EMP_NO, NAME FROM EMP WHERE SAL>5000;
```

```
CURSOR CUR_SAL IS  
SELECT * FROM EMP WHERE SAL>6570;
```

Explicit Cursors

- **Explicit Cursor (Syntax):**
- **Syntax for opening a cursor**

```
OPEN  cursor_name;
```

OPEN <CURSOR_NAME>;

For Example:

OPEN CUR_SAL;

Explicit Cursors

- **Explicit Cursor (Syntax):**
- **Syntax for fetching the data values**

```
FETCH cursor_name INTO [variable1, variable2, ...]  
                        / record_name];
```

FETCH <CURSOR-NAME> INTO <VAR1>,<VAR2>,<VAR3>...

OR

FETCH <CURSOR_NAME> INTO <RECORD-NAME>;

For Example:

FETCH CUR_SAL INTO mEMPNO, mEMPNAME;

FETCH CUR_SAL INTO EMP_REC;

Explicit Cursors

- **Explicit Cursor (Syntax):**
- **Syntax for closing a cursor area**

```
CLOSE      cursor_name;
```

CLOSE <CURSOR-NAME>;

For Example:

CLOSE CUR_SAL;

Explicit Cursor

- **Explicit Cursor (Example):**

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    c_id customers.id%type;
```

```
    c_name customers.name%type;
```

```
    c_addr customers.address%type;
```

```
    CURSOR c_customers IS SELECT id, name, address FROM customers;
```

```
BEGIN
```

```
    OPEN c_customers;
```

```
    LOOP
```

```
        FETCH c_customers into c_id, c_name, c_addr;
```

```
        EXIT WHEN c_customers%notfound;
```

```
        dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
```

```
    END LOOP;
```

```
    CLOSE c_customers;
```

```
END;
```

```
/
```

Explicit Cursor

- **Explicit Cursor (Example Output):**

1	Ramesh	Ahmedabad
2	Khilan	Delhi
3	kaushik	Kota
4	Chaitali	Mumbai
5	Hardik	Bhopal
6	Komal	MP

PL/SQL procedure successfully completed.

Comparison between Implicit and Explicit Cursors OR

Difference between Implicit and Explicit Cursors

IMPLICIT CURSOR	EXPLICIT CURSOR
(1). If Oracle engine for its internal processing has opened a cursor they are known as Implicit Cursors.	(1). A user can also open a cursor for processing data as required. Such user-defined cursors are known as Explicit Cursors.
(2). Oracle by default creates the cursor with name SQL.	(2). User can create the explicit cursor with any name and also can changes the name of the cursor.
(3). We have to put SQL prefix before accessing any attributes of the Implicit Cursor.	(3). If we want to use any Explicit Cursors attributes then we have to put user defined cursor's name as a prefix.
(4). User cannot require OPENING and CLOSING of the implicit cursor.	(4). User must have to OPEN the cursor for fetching the data and CLOSE the cursor after completion of operation.
(5). We cannot use the FETCH command with Implicit Cursor.	(5). We have to use the FETCH command with Explicit Cursor.
(6). There are no other types of implicit cursors.	(6). There are two types of explicit cursors are there: (i) Normal Cursor. (ii) Parameterized Cursor.

Cursor FOR Loop

- CURSOR FOR LOOP is used when you want to fetch and process every record in a cursor.
- The cursor FOR loop can be used to process multiple records.
- The advantage of cursor FOR loop is that the loop itself will OPEN a cursor, FETCHES (reads) the records into the cursor from the table until EOF is encountered, then it CLOSES the cursor.
- The CURSOR FOR LOOP will terminate when all of the records in the cursor have been fetched.
- # The VARIABLE used in the cursor FOR loop is not necessary to declare. It will be automatically created by the cursor FOR loop.

Cursor FOR Loop

- **Cursor FOR Loop Syntax:**

```
FOR <VARIABLE> IN <CURSOR_NAME> LOOP  
    <STATEMENTS>;  
END LOOP;
```

```
FOR record_name IN cursor_name LOOP  
    statement1;  
    statement2;  
    . . .  
END LOOP;
```

Cursor FOR Loop

- **Explicit Cursor (Example of Cursor FOR Loop):**

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    item emp3%rowtype;
```

```
    CURSOR c1 IS SELECT last_name, job_id FROM emp3 where job_id LIKE  
    '%CLERK%' AND manager_id > 120 ORDER BY last_name desc;
```

```
BEGIN
```

```
    FOR item IN c1 LOOP
```

```
        DBMS_OUTPUT.PUT_LINE ('Name = ' || item.last_name || ', Job = '  
        || item.job_id);
```

```
    END LOOP;
```

```
END;
```

```
/
```

Cursor FOR Loop

- **Output: (Example of Cursor FOR Loop):**

Name = Atkinson, Job = ST_CLERK

Name = Bell, Job = SH_CLERK

Name = Bissot, Job = ST_CLERK

...

Name = Walsh, Job = SH_CLERK

Parameterized Cursor

- Parameterized cursor pass the parameters into a cursor and use them in to query.
- Define only datatype of parameter and not need to define it's length.
- We can only pass values to the cursor and cannot pass values out of the cursor through parameters.
- The scope of the cursor parameters is local to the cursor.

Parameterized Cursor

- **Parameterized Cursor (Syntax):**
- **Syntax to declare Parameterized Cursor:**

```
CURSOR cursor_name  
    [(parameter_name datatype, ...)]  
IS  
    select_statement;
```

```
CURSOR <CURSOR_NAME>  
    [(PARAMETER1 DATATYPE, PARMETER2 DATATYPE,....)]  
IS <SELECT STATEMENT>;
```

- **Syntax to open Parameterized Cursor:**

```
OPEN  cursor_name(parameter_value, .....) ;
```

Parameterized Cursor

- **Parameterized Cursor (Example):**

```
CURSOR c2 (subject_id_in IN varchar) IS  
SELECT course_number FROM courses_tbl  
WHERE subject_id = subject_id_in;
```

```
OPEN C2(1);
```

Output:

- The result set of this cursor is all course_numbers whose
- subject_id matches the subject_id passed to the cursor via the parameter.

Advantages

- Using Cursor we can perform row by row processing .
- We can perform row wise validation or operations on each row.
- Allow application to access and move around in a set of data rows, rather than merely retrieve a complete result set.

Disadvantages

- Uses more resources.
- Speed and performance issues.
- Increased network roundtrip.